EXPRESS JS

JWT

Theory Notes

1. Express.js

- **Express** is a Node.js framework for building APIs and web servers.
- `express.Router()` allows modular route definitions.
- Middleware functions (`app.use(...)`) can process requests before reaching routes.

2. MySQL Integration

- Using `mysql2/promise` with `createPool()` creates a connection pool for efficient queries.
- Queries are executed with `await db.query("SQL", [params])`.
- Data is stored in a relational database (`user` table in MySQL).

3. Password Hashing

- **bcrypt** is used to hash passwords before storing them.
- `bcrypt.hash(password, saltRounds)` → generates a secure hash.
- `bcrypt.compare(plain, hash)` → verifies login credentials.
- Storing plain text passwords is insecure; hashing prevents exposure.

4. JWT Authentication

- **JWT (JSON Web Token)** is used for stateless authentication.
- `jwt.sign(payload, secret, options)` → creates a token.
- `jwt.verify(token, secret)` → validates token.
- Tokens are sent in the `Authorization` header.
- Middleware (`authMiddleware`) checks token validity before allowing access.

5. REST API Endpoints

- `POST /register` → creates a new user with hashed password.
- `POST /login` → verifies credentials, returns JWT token.
- `GET /users` → fetches all users.
- `GET /profile` → protected route, requires valid JWT.

6. Environment Variables

- `.env` file stores sensitive data like `PORT`, `DB_PASSWORD`, `JWT_SECRET`.
- `process.env.PORT` → loads port dynamically.
- Prevents hardcoding secrets in code.

Practice Tasks

Short Answer

1. Why is bcrypt used instead of storing plain text passwords?
2. What is the difference between `jwt.sign()` and `jwt.verify()`?
3. Why should secrets (like JWT keys) be stored in `.env` files?
4. What happens if you don't call `next()` in middleware?

Coding Exercises

1. Add an `email` field to the `user` table and update registration/login to use email instead of name.
2. Modify `authMiddleware` to check for `Bearer <token>` format in the `Authorization` header.
3. Add a `role` field (e.g., `"admin"`, `"user"`) and create middleware that restricts access to certain routes based on role.
4. Implement a `PUT /user/:id` route to update user details.
5. Add error handling middleware that returns JSON `{ error: "message" }` for invalid routes.
6. Store JWT secret in `.env` and load it with `process.env.JWT_SECRET`.
7. Add token refresh functionality (`/refresh`) to issue new tokens when old ones expire.

```
const express = require("express");
require("dotenv").config();
const userRoutes = require("./routes/userRoutes")

const app = express();

app.use(express.json());

app.use(userRoutes);
app.listen(process.env.PORT, () => {
  console.log(`Server is up and running port ${process.env.PORT}`);
});
```

Db.js

```
const mysql = require("mysql2/promise");

const pool = mysql.createPool({
  host: "localhost",
  user: "root",
  password: "raj123",
  database: "puru",
});
```

```
module.exports = pool;
```

userController.js

```
const db = require("../db");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");

const registerUser = async (req, res) => {
  const { name, password } = req.body;

  const hashedPassword = await bcrypt.hash(password, 10);

  await db.query("INSERT INTO user (name, password) VALUES( ?,?) ", [
    name,
    hashedPassword,
  ]);
  res.json("User created successfully");
};

const login = async (req, res) => {
  const { name, password } = req.body;

  const [rows] = await db.query("SELECT * FROM user WHERE name = ?", [name]);

  if (rows.length === 0) {
    return res.send("username not found please register");
  }

  const user = rows[0];
  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch) return res.send("Password not match");

  const token = jwt.sign({ id: user.id }, "secretkey", { expiresIn: "1h" });

  res.json({
    user,
    token,
  });
};


const authMiddleware = async (req,res,next )=>{
  const token = req.headers["authorization"]
  console.log(req.headers);
  console.log(req.headers.authorization);
```

```
  if (!token) return res.json({ message: "Access denied" });
  try {
    const verified = jwt.verify(token, "secretkey");
    req.user = verified;
    next();
  } catch (err) {
    res.status(400).json({ message: "Invalid token" });
  }
};

const getUser = async (req, res) => {
  const [rows] = await db.query("SELECT * FROM user");

  res.json(rows);
};

module.exports = {
  registerUser,
  getUser,
  login,
  authMiddleware
};
```

userRoutes.js

```
const express = require("express");
const {
  registerUser,
  getUser,
  login,
  authMiddleware,
} = require("../controller/userController");

const router = express.Router();

router.post("/register", registerUser);
router.get("/users", getUser);
router.post("/login", login);

router.get("/profile", authMiddleware, (req, res) => {
    console.log(req.header);

  res.send("AUth middleware working fine ");
});
module.exports = router;
```

Theory Notes

1. Express.js Setup

- `express()` creates the server.
- `express.json()` parses JSON request bodies.
- Routes are modularized in `userRoutes`.
- `app.listen(process.env.PORT)` starts the server on port defined in `.env`.

2. MySQL Integration

- Using `mysql2/promise` with `createPool()` creates a connection pool.
- Queries are executed with `await pool.query("SQL", [params])`.
- Pooling improves performance by reusing connections.

3. Password Hashing

- **bcrypt** is used to hash passwords before storing them.
- `bcrypt.hash(password, 10)` → generates a secure hash.
- `bcrypt.compare(plain, hash)` → verifies login credentials.
- Storing plain text passwords is insecure; hashing prevents exposure.

4. JWT Authentication

- **JWT (JSON Web Token)** is used for stateless authentication.
- `jwt.sign(payload, secret, { expiresIn })` → creates token.
- `jwt.verify(token, secret)` → validates token.
- Tokens are sent in the `Authorization` header.
- Middleware (`authMiddleware`) checks token validity before allowing access.

5. REST API Endpoints

- `POST /register` → creates a new user with hashed password.
- `POST /login` → verifies credentials, returns JWT token.
- `GET /users` → fetches all users.
- `GET /profile` → protected route, requires valid JWT.

6. Environment Variables

- `.env` file stores sensitive data like `PORT`, `DB_PASSWORD`, `JWT_SECRET`.
- Prevents hardcoding secrets in code.
- Example:

env
```
PORT=3000
JWT_SECRET=mysecretkey
```